Guía del paquete de reproducibilidad (para Mónica)

Esta guía explica, en palabras simples, qué hay en la carpeta `reproducibility/`, para qué sirve cada programa, cómo se ejecuta y cómo subirlo a Zenodo. El manual oficial, en inglés, para editores y revisores es `reproducibility/README.md`; esta guía es para que tú puedas entenderlo y defenderlo.

---

1. Qué es el paquete

Es una carpeta autocontenida con todo lo que respalda el paper:

Carpeta	Qué contiene	En palabras simples
`analysis/`	los programas del método y de cada análisis (Python)	lo que "calcula"
`results/`	los resultados de esos cálculos, un archivo por problema (3 MB)	los números del paper
`figures_tables/`	los programas que hacen cada figura, tabla y nota del suplemento	lo que "dibuja"
`lean_proofs/`	las demostraciones en Lean 4 (32 módulos, 304 teoremas y lemas, 28 modelos)	lo que "demuestra"
`lean_translation/`	el traductor SBML → Lean y la comprobación de la traducción	el puente entre modelos y Lean
`scripts/`	descarga del benchmark y comparación de resultados	utilidades

No incluye: los problemas PEtab (se descargan con `scripts/get_benchmark.sh`) ni los datos de pasos del participante del modelo SCT (se entregan "a pedido razonable", como dice el paper).

---

2. Qué hace cada programa (en orden de uso)

Programa (`analysis/`)	Qué hace	Produce
`config.py`	guarda todos los umbrales del paper (δ = 1 %, τ_R = 0,89, κ/VIF ≤ 10, τ_e = 0,436, 15 escenarios, semilla 42)	—
`petab_problem.py`	simula las **salidas medidas** de un problema PEtab: y_i = h(g_i(x(t_i), θ))/σ_i	—
`simulator_check.py`	compara nuestra simulación con la del benchmark	Tabla S9
`selection.py`	**el método**: sensibilidad J, etapa 1 (energía), etapa 2 (efecto finito), κ/VIF, admisibilidad con e_fit, poda	Fig. 2, Tabla 1, Tabla S1
`selection.py --variant …`	lo mismo cambiando un ajuste (δ, tolerancias, ±1 %/±10 %, τ_R, κ/VIF, τ_e)	Fig. 5, Tabla S14

Programa (`analysis/`)	Qué hace	Produce
`external_validation.py`	ajusta con unos experimentos y predice los otros (datos reales y sintéticos)	Fig. 3, Fig. S1, Tabla S12
`synthetic_seeds.py`	experimento sintético con 20 semillas; con `SINT_SOLO_SENAL=1` calcula solo la señal $y(\theta^*)-y(\theta_0)$ en σ, y con `SINT_AMPLITUD=0.05/0.50` repite el experimento con otras amplitudes (punto C3)	Fig. 3b; tabla y figura de señal y amplitudes
`robustness.py`	mide el error a ±1…50 % con las mismas 15 direcciones; patrones A–D	Fig. 5, Tablas S10–S11, Fig. S2
`robustness.py` con `ROBUST_INDEP=1`	lo mismo con direcciones **nuevas** en cada tamaño (punto B5 de la auditoría)	tabla y figura de direcciones independientes (nota de robustez)
`absolute_error.py`	expresa el error que deja el reducido en unidades del ruido σ (punto C1)	tabla del error en σ (nota de ajustes)
`single_parameter.py`	prueba si un solo parámetro bastaría (justifica k_min = 2)	Tabla S13
`crauste_blowup.py`	muestra por qué Crauste no tiene reducción local	texto de Resultados
`timing.py`	mide el tiempo de una simulación	Tabla S15
`sct_model.py`, `sct_analysis.py`	el ejemplo SCT (necesita los datos del participante)	Tabla S16, Fig. S3

Programa (`lean_translation/`)	Qué hace
`sbml_to_lean.py`	traduce cada modelo SBML a un archivo Lean con números racionales exactos
`verify_translation.py`	comprueba, sin usar el traductor, que el archivo Lean es el mismo modelo que el SBML (diferencia $\sim 10^{-12}$)
`mutation_control.py`	control: si se cambia un número del archivo Lean en 1 %, la comprobación lo detecta ($\sim 10^{-2}$)

`figures_tables/build_all.py` ejecuta todos los generadores de figuras, tablas y notas; entre ellos `make_review_notes.py`, que produce las tablas y figuras de los análisis adicionales (direcciones independientes, señal y amplitudes, error en σ y variantes de ajustes de Lang y Raimundez), y `make_timing_notes.py`, que reúne el tiempo de cómputo de cada análisis en cada problema (campo `segundos` de los resultados) y produce las tablas de la nota "Computing time of the analyses" del suplemento; su objetivo es mostrar que seleccionar y verificar un subconjunto es asequible (minutos en casi todos los problemas, mucho menos que la verosimilitud perfilada del modelo completo, que no fue viable en los problemas grandes). `figures_tables/cifras_texto.py` recalcula las cifras que cita el texto principal (admisibles, identificabilidad, validación, comparación de criterios, robustez) para comprobarlas tras cualquier cambio.

---

3. Cómo ejecutarlo en tu computadora

Necesitas Python 3.11. En una terminal, dentro de la carpeta `reproducibility/`:

```
python -m venv .venv
.venv\Scripts\activate          (en Windows; en Mac/Linux: source .venv/bin/activate)
pip install -r requirements.txt
```

Nivel A (menos de un minuto): rehacer figuras y tablas desde los resultados guardados

```
cd figures_tables
python build_all.py
```

Aparecen en `figures_tables/output/`. Para Fig. 6 y las copias EPS/TIFF hacen falta `pdflatex` y poppler (`pdftops`, `pdftoppm`).

Nivel B (1–2 horas): compilar Lean

Instalar elan (https://github.com/leanprover/elan) y luego, dentro de `lean_proofs/`: `./verify.sh` (en Windows, desde Git Bash o WSL).

Nivel C (1–2 días en total): recalcular desde cero

```
bash scripts/get_benchmark.sh
set REPRO_RESULTS=%CD%\results_rerun      (Windows cmd; en PowerShell: $env:REPRO_RESULTS="$PWD\results_
cd analysis
python selection.py Boehm_JProteomeRes2014
cd ..
python scripts/compare_results.py results_rerun/selection results/selection
```

Puedes recalcular un solo problema (como en el ejemplo) o todos (sin nombres). Los pasos y tiempos están en la tabla del README.

---

4. Qué comprobé antes de entregártelo

- **Código equivalente.** Los programas del paquete son versiones ordenadas (en inglés) de los que usamos (`certificados/`). Comprobé automáticamente que la lógica es la misma, comparando la estructura del código función por función, y además volví a ejecutarlos y comparé los resultados con los guardados (ver la sección 6).
- **Traducción a Lean.** Al volver a ejecutar el traductor, los 28 archivos de modelos salen **idénticos** a los que se compilaron.
- **Figuras y tablas.** En un entorno de Python nuevo, instalado solo con `requirements.txt`, el nivel A regenera todo en unos 20 segundos, y las tablas, las figuras y las notas coinciden con las del paper.
- **Benchmark.** `get_benchmark.sh` descarga exactamente los mismos archivos que usamos (commit fcbddf1).

---

5. Correcciones que salieron al armar el paquete

Al rehacer todo con cuidado aparecieron cuatro cosas que corregí en el paper:

1. **Prueba de un solo parámetro.** La validación externa del mejor parámetro único no reajustaba los parámetros de calibración (escalas, offsets), mientras que la del subconjunto S sí. Lo recalculé de forma justa (con la calibración, igual que S) y actualicé la Tabla S13 y la frase del paper.
2. **Crauste.** El paper decía que con ρ_E −1 % los efectores pasan de 5×10^2 a 5×10^5 en cuatro días, pero eso también ocurre con los parámetros publicados. Lo correcto: con los parámetros publicados los efectores llegan a ~10^6 cerca del día 10 y luego bajan; con un cambio del 1 % siguen creciendo y la solución se rompe entre los días 8,8 y 9,9. Ocurre en 9 de las 24 perturbaciones (antes decía "8–9") y en 7 de 15 escenarios, con cualquier tolerancia.

3. **Robustez: una sola corrida.** Había dos corridas de robustez (la segunda guarda cada escenario). Ahora todo sale de la segunda. Solo cambia Chen, que en Methods pasa de "entre 7 y 13 escenarios" a "entre 8 y 13"; los patrones y todas las demás cifras son iguales.
4. **Rutas.** El paper y el suplemento citaban programas de `certificados/`; ahora citan el paquete.

---

6. Cómo subirlo a Zenodo (paso a paso)

1. Entra a https://zenodo.org con tu cuenta (puedes usar ORCID o GitHub).
2. **New upload**.
3. Arrastra el archivo `reproducibility_v1.0.0.zip` (está en la carpeta `zenodo/` del repositorio).
4. Pulsa **Reserve DOI** (o "Get a DOI now"): Zenodo te muestra el DOI, algo como `10.5281/zenodo.1234567`.
5. Completa los datos:
   - *Resource type*: Software.
   - *Title*: "Reproducibility package: Few biological parameters suffice to reproduce measured dynamics in systems biology models".
   - *Creators*: los cuatro autores, con afiliación ESPOL (y ORCID si los tienen).
   - *Description*: puedes copiar el primer párrafo del README.
   - *License*: MIT.
   - *Version*: 1.0.0.
6. **Antes de publicar**, pon el DOI en el paper. En `main.tex` y `supplementary.tex` (arriba del todo) reemplaza `10.5281/zenodo.XXXXXXX` por tu DOI en la línea `\newcommand{\zenododoi}{...}`; con eso se actualizan todas las menciones. Compila de nuevo.
7. Publica en Zenodo. Si después hay cambios (por ejemplo tras la revisión), se sube una **nueva versión** del mismo registro y Zenodo le da otro DOI, manteniendo el historial.

Si prefieres no publicar todavía, Zenodo permite reservar el DOI y dejar el registro en borrador.

---

7. Si un revisor pregunta…

Pregunta	Respuesta y dónde está
¿Cómo reproduzco una cifra?	README, tabla "Where each result comes from": cifra → programa → archivo de resultados
¿Qué significa un campo del JSON?	`DATA_DICTIONARY.md`
¿Son deterministas los resultados?	Sí: semillas fijas. Solo Chen puede variar levemente (algunas simulaciones fallan al integrar)
¿Por qué hay nombres en español en los resultados?	Son los nombres originales de los programas; el diccionario los traduce
¿El archivo Lean es de verdad el modelo del benchmark?	`verify_translation.py` (diferencia $\sim 10^{-12}$) y el control del 1 %
¿Por qué no está el modelo completo de Chen y Raimundez en el sintético?	Tienen más de 100 parámetros libres; con datos reales sí se reajusta (validación anidada)
¿Por qué Lang no es admisible?	La búsqueda voraz no llega a un par admisible con los ajustes por defecto, aunque existen (nota *Random scenarios and large problems*, tabla de variantes de Lang)
¿Cómo se tratan los parámetros negativos o nulos en los escenarios?	Perturbación multiplicativa sin recorte: conservan el signo; los nulos se representan por 10^{-12} y nunca se eligen (misma nota)
¿Se obtiene lo mismo en otra máquina?	Sí: la selección de Lang y Raimundez coincidió con libRoadRunner 2.9.2 y 2.10.0 (e_fit iguales hasta 10^{-8})

Pregunta	Respuesta y dónde está
¿Dónde están los datos del SCT?	No son públicos; se entregan a pedido razonable